

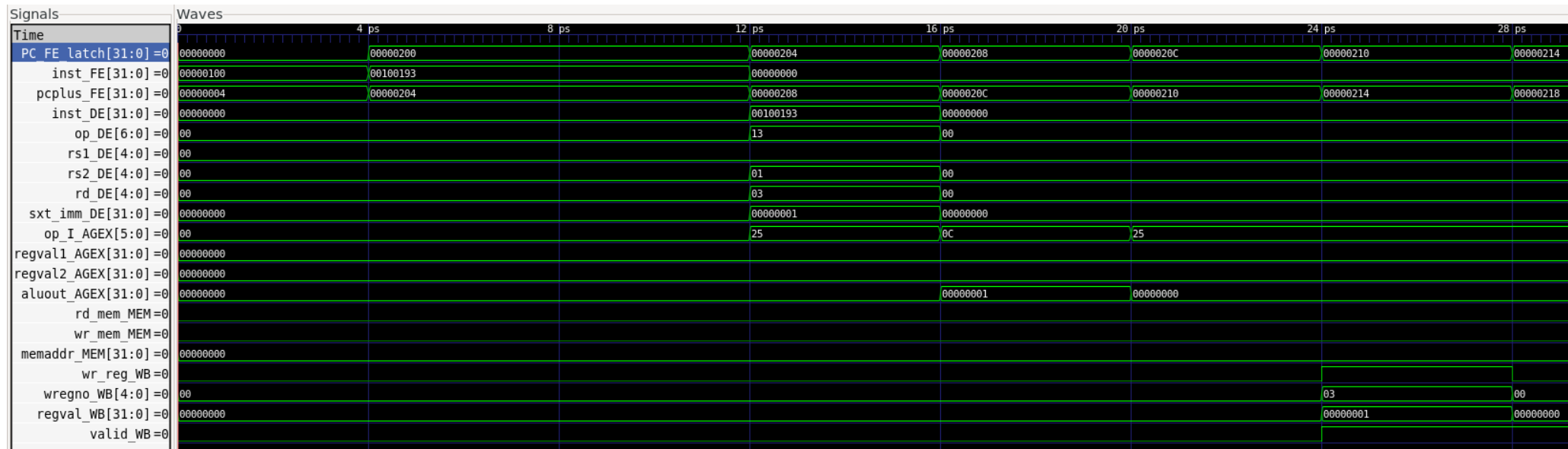
Devin Fromond

CS 3220 | A

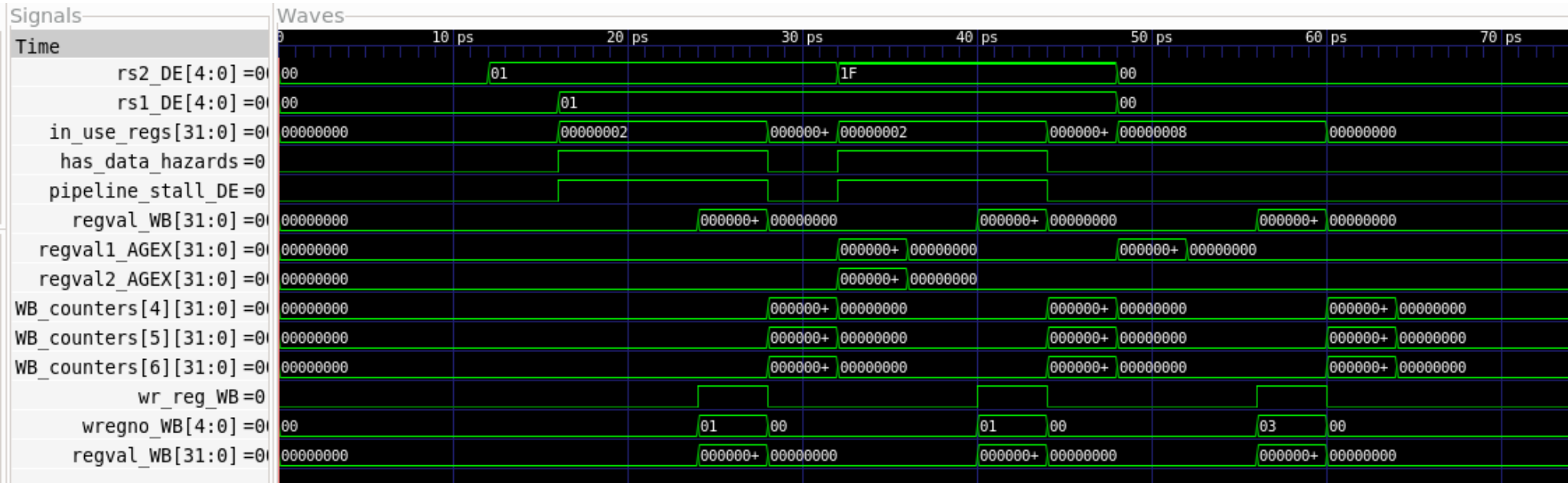
Lab 01

1/28/2026

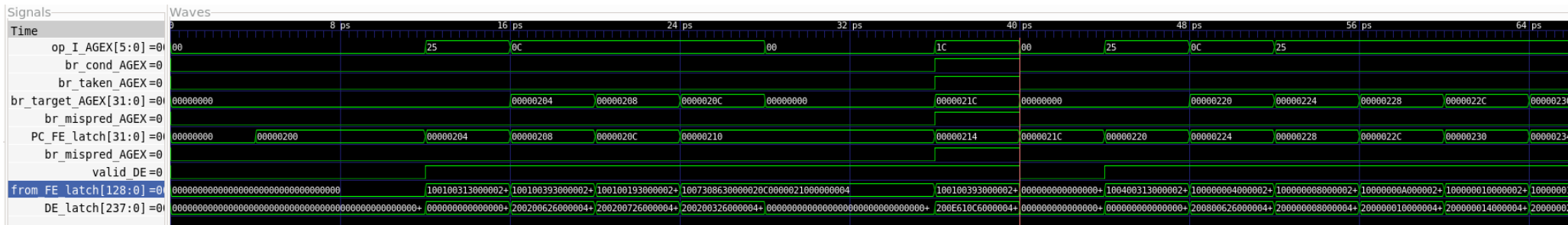
Part 1: Minimal Functionality



In executing `test/part1/test1.mem`, FE fetches sequential PCs, beginning with `0x0200`; DE decodes the ADDI instruction and pulls `x0` and the immediate value; AGEX forwards `regval1=0`, `stx_imm=imm` into the ALU for addition; MEM (quite trivially) passes through since there is no load/store instruction; WB asserts `wr_reg_WB=1` and writes the ALU result into the destination register. The screenshot above highlights how the signals move through the pipeline, emphasizing how there is no MEM signals flipped and `op_DE` contains the opcode for ADDI (`0x13`).



The execution of `test/part1/test2.mem` shows back-to-back instructions where `x6` consumes `x5`'s result and so on. In the cycle where `add x6` reaches AGEX, `regval1_AGEX` pipes its data from the MEM/WB forward path. Since forwarding succeeds, `pipeline_stall_DE` and `has_data_hazards` are low as possible with the provided pipeline. We are able to see a direct connection between `in_use_regs` and the `pipeline_stall_DE` and `has_data_hazards` flags, indicating that, during this time, there is a RAW hazard being processed. However, after those flip to low, we immediately observe `WB_counters` flip to high, indicating the register write is committing. This waveform highlights how the pipeline handles RAWs since it stalls younger instructions until the producer writes back, then continues once the dependency is cleared.



From the waveform above, we observe that for `test/part1/test4.mem` at the arbitrary cycle N, BEQ evaluates operands, `br_taken_AGEX=1` contradicts the not-taken prediction (at the point where the red line is), so `br_mispred_AGEX=1` pulses, `br_target_AGEX` holds the correct PC, FE uses it in cycle N=1, and the younger instruction in DE is flushed – which explains the period of low signals after 40ps.